



NODE TREE CONTRACT

Версия: 1.0.0

Дата: 2026-06-13

Статус: Активный контракт

Автор: Koda (NLP-Core-Team)

ЦЕЛЬ

Этот контракт определяет каноническую структуру дерева узлов экосистемы Balloo, обеспечивая возможность полного восстановления архитектуры по документации без потери архитектурного смысла.

Primary Purpose: Позволить AI-агенту без контекста чата:

1. Понять какие узлы существуют в экосистеме
 2. Определить сервисы, домены, роли и зависимости каждого узла
 3. Восстановить экосистему заново без потери архитектурного смысла
-

GLOSSARY

Логическая роль (Logical Role)

Функциональное назначение узла в архитектуре (control-plane, execution-plane, storage-plane).

Node State

- **Active** - узел работает и выполняет свои функции
- **Inactive** - узел выключен или временно недоступен
- **Optional** - узел не является обязательным для работы экосистемы

Source of Truth

Первоисточник истины для данных, конфигураций или состояния системы.

Control Plane

Слой управления, отвечающий за оркестрацию, deployment и администрирование.

Execution Plane

Слой выполнения рабочей нагрузки, где работают production, stage и dev приложения.

Storage Plane

Слой хранения данных для резервного копирования и архивирования.

🌳 КАНОНИЧЕСКОЕ ДЕРЕВО УЗЛОВ

```
ROOT
├── control-plane (Управление и оркестрация)
│   ├── laptop_control (Основной control node)
│   ├── phone_personal (Персональный доступ)
│   ├── phone_service (Служебный доступ)
│   └── phone_recovery_optional (Опциональный recovery)
├── execution-plane (Выполнение рабочих нагрузок)
│   ├── work_server (Primary production host)
│   └── home_aio (Secondary dev/stage node)
└── storage-plane (Хранение и резервирование)
    └── home_nas (Backup и хранение)
```

📋 ОПИСАНИЕ ГРУПП УЗЛОВ

control-plane

Purpose: Группа узлов управления и оркестрации

Why exists separate from execution nodes:

- Control plane НЕ является местом выполнения production workload
- Control plane содержит инструменты управления, а не бизнес-логику
- Разделение обеспечивает безопасность и изоляцию operator devices
- Control plane может быть временно недоступен без остановки production

Characteristics:

- MAY быть выключен без остановки production сервисов
- SHOULD иметь доступ ко всем узлам через SSH/API
- MUST содержать git client, VS Code/Koda, CLI tools
- MUST NOT выполнять production workload

Nodes:

- **laptop_control** - Primary control node (обязательный)
- **phone_personal** - Personal access endpoint (обязательный)
- **phone_service** - Service access endpoint (обязательный)
- **phone_recovery_optional** - Recovery endpoint (опциональный)

execution-plane

Purpose: Группа узлов выполнения рабочих нагрузок

Why exists separate from control plane:

- Execution plane НЕ является местом управления
- Execution plane содержит приложения и сервисы
- Разделение обеспечивает безопасность и изоляцию
- Execution plane может масштабироваться независимо

Characteristics:

- MUST быть всегда включён (always-on для production)
- MUST выполнять production/dev/stage workload
- SHOULD иметь резервное копирование
- Criticality: **CRITICAL** (production), **MEDIUM** (dev/stage)

Nodes:

- **work_server** - Primary production host (обязательный)
 - **home_aio** - Secondary dev/stage node (опциональный)
-

storage-plane

Purpose: Группа узлов хранения и резервирования

Why exists separate from execution nodes:

- Не является execution node
- Используется для backup/archive
- Хранит зашифрованные данные
- Обеспечивает восстановление системы

Characteristics:

- MAY быть выключен без остановки production
- MUST хранить encrypted backups
- MUST синхронизироваться с execution-plane
- MUST NOT выполнять production workload
- Criticality: **HIGH** (для recovery)

Nodes:

- **home_nas** - Backup и storage node (обязательный)
-

ОПИСАНИЕ УЗЛОВ

1. laptop_control

Canonical Name: **laptop_control**

Aliases: **laptop, control-node, operator-device**

OS Target: Windows 11 Pro / Linux

Role Class: control-node

Responsibilities:

- MUST быть главным entry point для orchestration
- MUST содержать VS Code / Koda для разработки
- MUST иметь git client для работы с репозиториями
- MUST иметь SSH client для доступа к узлам
- MUST использоваться для запуска deployment команд
- MUST использоваться для запуска audit команд
- MAY содержать локальные dev окружения

Boundaries:

- MUST NOT выполнять production workload
- MUST NOT хранить production данные без шифрования
- MUST NOT быть единственным местом хранения кода (git remote обязателен)
- MUST NOT запускать production database

Network & Access:

- Private access: Tailscale/SSH через execution-plane
- Public exposure: NONE (control node не публикует сервисы)
- Ingress: Только исходящие подключения

Deployment Role:

- Dev: YES (локальная разработка)
- Stage: MAY (preview environments)
- Prod: NO (не является execution node)
- Can initiate rollout: YES (главный control point)
- Can host production: NO

Recovery Role:

- Recovery priority: HIGH (главный operator interface)
- What breaks if absent: Невозможно управлять системой
- Recovery method: Замена устройства + восстановление git/SSH ключей

2. work_server

Canonical Name: work_server

Aliases: work, production-server, main-server

OS Target: Linux (Ubuntu 22.04 LTS / Debian 12)

Role Class: production-node

Responsibilities:

- MUST быть primary host для production приложений
- MUST выполнять backend/API сервисы
- MUST хранить production database (PostgreSQL)
- MUST выполнять websocket/realtime сервисы
- MUST выполнять reverse proxy (Nginx/Caddy)
- MAY выполнять Ollama heavy AI модели
- MAY выполнять Open WebUI
- MUST быть source of truth для deployment metadata

Domain Binding:

- **balloo.su** - messenger ecosystem (ПУБЛИЧНЫЙ)

Services MUST live here:

- API Server (Express.js)
- PostgreSQL database
- Redis cache
- WebSocket server
- Reverse proxy
- Production Messenger app
- Production Admin Portal
- Ollama (heavy models)

Services MUST NOT live here:

- Local dev environments
- Personal files
- Non-encrypted sensitive data

Services MAY live here:

- Stage/preview environments (isolated)
- Monitoring (Prometheus/Grafana)
- CI/CD runners (GitHub Actions self-hosted)

Network & Access:

- Private access: Tailscale, internal network
- Public exposure: Через reverse proxy только нужные сервисы
- Ingress: HTTP/HTTPS через reverse proxy
- Egress: Internet для API, npm, Docker Hub

Deployment Role:

- Dev: NO
- Stage: MAY (изолированные preview)
- Prod: YES (единственный production host)
- Can initiate rollout: NO (принимает команды от control-plane)
- Can host production: YES (единственный узел)

Recovery Role:

- Recovery priority: CRITICAL (самый важный узел)
 - What breaks if absent: BCE production падает
 - Recovery method: Восстановление из backup + развертывание на новом узле
 - Backup source: home_nas, git repositories
-

3. home_aio

Canonical Name: home_aio

Aliases: home, aio, secondary-dev

OS Target: Linux (Ubuntu 22.04 LTS)

Role Class: dev-node

Responsibilities:

- MAY быть secondary dev environment
- MAY запускать preview environments
- MAY выполнять secondary AI model inference
- MAY использоваться для local CI/validation
- MAY быть staging helper
- MAY зеркалить код с work_server

Boundaries:

- MUST NOT быть production source of truth
- MUST NOT хранить production данные без явного бэкапа
- SHOULD синхронизироваться с work_server
- MUST быть изолирован от production traffic

Network & Access:

- Private access: Tailscale
- Public exposure: MAY (preview environments через tunnel)
- Ingress: Только из private network

Deployment Role:

- Dev: YES (основная dev среда)
- Stage: YES (preview/staging)
- Prod: NO (не production)
- Can initiate rollout: NO (только control-plane)
- Can host production: NO

Recovery Role:

- Recovery priority: LOW (некритичен для production)
- What breaks if absent: Dev workflow замедляется
- Recovery method: Восстановление из git + sync с work_server

4. home_nas

Canonical Name: home_nas

Aliases: nas, backup-server, storage

OS Target: macOS / Linux (с NETATALK/SMB)

Role Class: backup-node

Responsibilities:

- MUST хранить backups work_server
- MUST хранить git/artifact mirrors
- MAY хранить media файлы
- MAY выполнять non-critical задачи
- MAY быть fallback proxy/docs mirror
- MUST быть резервным узлом для восстановления

Boundaries:

- MUST NOT быть primary execution node для production
- MUST NOT выполнять production workload
- SHOULD быть только для хранения и зеркалирования
- MUST хранить зашифрованные бэкапы

Network & Access:

- Private access: Tailscale, local network
- Public exposure: NONE
- Ingress: Только backup sync от work_server

Deployment Role:

- Dev: MAY (хранилище артефактов)
- Stage: NO
- Prod: NO
- Can initiate rollout: NO
- Can host production: NO

Recovery Role:

- Recovery priority: HIGH (хранилище бэкапов)
- What breaks if absent: Невозможно восстановить work_server
- Recovery method: Замена NAS + восстановление из remote backup

5. phone_personal

Canonical Name: phone_personal

Aliases: personal-phone, user-phone

OS Target: Android / iOS

Role Class: access-node

Responsibilities:

- MAY использоваться для access к сервисам
- MAY получать notifications
- MUST использоваться для auth confirmation (2FA)
- MAY использоваться для quick checks

Boundaries:

- MUST NOT быть execution node
- MUST NOT запускать серверные сервисы
- MUST NOT хранить production данные

Network & Access:

- Private access: Tailscale (опционально)
- Public exposure: N/A (клиентское устройство)
- Ingress: Push notifications

Deployment Role:

- Dev: NO
- Stage: NO
- Prod: NO
- Can initiate rollout: NO
- Access to production: YES (через browser/app)

Recovery Role:

- Recovery priority: MEDIUM (потеря доступа)
- What breaks if absent: Невозможно пройти 2FA
- Recovery method: Привязка нового устройства + recovery codes

6. phone_service

Canonical Name: phone_service

Aliases: service-phone, admin-phone

OS Target: Android / iOS

Role Class: access-node

Responsibilities:

- MUST использоваться для isolated service access
- MUST использоваться для second-factor admin checks
- MUST использоваться для project separation
- MAY использоваться для admin 2FA

Boundaries:

- MUST NOT быть личным устройством
- MUST быть отделён от personal phone
- MUST NOT использоваться для личных целей

Network & Access:

- Private access: Tailscale (опционально)
- Public exposure: N/A
- Ingress: Push notifications для admin alerts

Deployment Role:

- Dev: NO
- Stage: NO
- Prod: NO
- Can initiate rollout: NO
- Access to admin: YES (изолированный доступ)

Recovery Role:

- Recovery priority: MEDIUM (admin access)
- What breaks if absent: Сложнее делать admin checks
- Recovery method: Привязка нового устройства для admin access

7. phone_recovery_optional

Canonical Name: phone_recovery_optional

Aliases: recovery-phone, rugged-phone

OS Target: Android / iOS

Role Class: recovery-node

Responsibilities:

- MAY использоваться как rugged recovery endpoint
- MAY использоваться для emergency access
- SHOULD быть всегда заряжен и готов

Boundaries:

- MUST быть по умолчанию inactive
- MUST NOT участвовать в основном deployment path
- MUST NOT использоваться для повседневных задач

Network & Access:

- Private access: Tailscale
- Public exposure: N/A
- Ingress: Emergency access только

Deployment Role:

- Dev: NO
- Stage: NO
- Prod: NO
- Can initiate rollout: NO
- Access in emergency: YES

Recovery Role:

- Recovery priority: LOW (опционально)
- What breaks if absent: Нет emergency access device
- Recovery method: Включение устройства + настройка access

RULES OF OWNERSHIP

Naming Convention

Формат: <plane>-<role>[_<modifier>]

Примеры:

- `laptop_control` - control plane, control role
- `work_server` - execution plane, server role
- `home_aio` - execution plane, aio role
- `home_nas` - storage plane, backup role

Правила:

- MUST использовать lowercase
- MUST использовать underscore как разделитель
- MUST явно указывать plane в имени
- SHOULD указывать role/функцию

Active/Inactive/Optional States

Active:

- `laptop_control` - MUST быть active когда выполняется управление
- `work_server` - MUST быть всегда active (always-on)
- `phone_personal` - SHOULD быть active для 2FA

Inactive:

- `home_aio` - MAY быть выключен когда не используется
- `home_nas` - MAY быть выключен когда не делается backup

Optional:

- `phone_service` - MAY не использоваться если нет separate admin access

- `phone_recovery_optional` - MAY не существовать вообще

Required Invariants

1. Control Plane Invariant:

- MUST иметь хотя бы один active control node
- `laptop_control` MUST быть доступен для deployment

2. Execution Plane Invariant:

- MUST иметь хотя бы один active production node
- `work_server` MUST быть доступен для production
- `work_server` MUST иметь backup на home_nas

3. Storage Plane Invariant:

- home_nas MUST хранить актуальные бэкапы
- home_aio MAY быть недоступен без влияния на production

4. Network Invariant:

- BCE узлы MUST иметь доступ к Tailscale network
- BCE production сервисы MUST быть доступны через reverse proxy

5. Domain Invariant:

- `balloo.su` MUST указывать на work_server
- Internal сервисы MUST НЕ быть публично доступны

RECOVERY IMPLICATIONS

Recovery Order (при полной потере системы)

Tier 1 (Критично):

1. `laptop_control` - восстановить access
2. `work_server` - восстановить production
3. `home_nas` - восстановить backups

Tier 2 (Важно):

4. `phone_personal` - восстановить 2FA access
5. `home_aio` - восстановить dev environment

Tier 3 (Опционально):

6. `phone_service` - восстановить admin access
7. `phone_recovery_optional` - восстановить emergency access

What Each Document Must Enable

NodeTreeContract.md:

- Понять дерево узлов и их иерархию
- Определить роли и зависимости

NodeRolesContract.md:

- Понять responsibilities каждого типа узла
- Определить allowed/forbidden services

NodeRolesContract.md:

- Понять responsibilities каждого типа узла
- Определить allowed/forbidden services

NodeDomainsContract.md:

- Понять доменные привязки
- Определить public/private boundaries

NodeRecoveryContract.md:

- Знать порядок восстановления
- Понять recovery priorities

NodeDeploymentContract.md:

- Понять deployment flow
- Определить source of truth

☒ CRITERIA FOR RECONSTRUCTION AI

AI-агент, восстанавливающий экосистему, ДОЛЖЕН:

1. **Прочитать все node contracts**
2. **Понять физическую и логическую topology**
3. **Определить node responsibilities для каждого узла**
4. **Восстановить canonical tree без ошибок**
5. **Не спутать dev/stage/prod окружения**
6. **Не спутать physical устройства и logical роли**
7. **Не потерять доменные привязки**
8. **Создать все state files (JSON)**
9. **Создать все topology maps**

Создано: 2026-06-13

Версия: 1.0.0

Статус: Active Contract

Автор: Koda (NLP-Core-Team)

 **Balloo - Share your moments safely!**

